Efficient Dynamic Proof of Retrievability for Cold Storage

Tung Le[‡], Pengzhi Huang[*], Attila A Yavuz[§], Elaine Shi[†], Thang Hoang[‡1]

[‡]Virginia Tech, [*]Cornell, [§]USF, [†]CMU

to appear in NDSS 2023

[1] https://thanghoang.github.io

Outline

Proof of Retrievability

Incrementally Constructible Code

DPoR Scheme [SSP13]

Our DPoR Scheme: Porla

Table of Contents

Proof of Retrievability

Incrementally Constructible Code

DPoR Scheme [SSP13]

Our DPoR Scheme: Porla

Proof of Retrievability (PoR)

Storage-as-a-Service (STaaS) offers remote data storage facility for individual and enterprise users.

Data security concerns in STaaS:

1. Integrity
2. Retrievability

Existing technologies address integrity and retrievability in STaaS:

1. Provable Data Possession (PDP) (not provable security)
2. **Proof of Retrievability (PoR)** (provable security)

PoR Overview

Existing PoRs rely on coding theory and cryptography techniques as building blocks

- Error Correction Code (ECC)
- Oblivious RAM (ORAM)
- Verifiable Computation (VC)

Most efficient PoR schemes are based on ECC to achieve low asymptotic complexity for all data operations (read/write/audit).

ECC-based PoR

Let (m, n, d) be the ECC parameters, where $m > n$ is the codeword length, n is the data length, and d is error tolerance capacity.

The **high-level idea** is to encode data blocks $(b_1, \ldots, b_n)$ with (m, n, d)-ECC as

$$(c_1, \ldots, c_m) \leftarrow \text{ECC.Encode}(b_1, \ldots, b_n)$$

and check the integrity of λ random codewords.

ECC-based PoR exploit the following facts (proven in [JK07])

- If adversary only deletes a small portion of data ($\leq d$), ECC can help to recover the original data.
- Otherwise, if adversary deletes large portion of data ($> d$), checking λ random positions will hit at least one deleted position (with overwhelming probability).

ECC-based PoR

ECC-based PoR does not support dynamic update naturally.

An update on d_i results in rebuilding the entire codeword.

- Complexity: $O(n)$

Data update may happen in practice on a regular basis.

Solution: Locally Updatable ECC (LU-ECC)

ECC-based DPoR

[SSP13] proposed a ECC-based Dynamic PoR (DPoR) by designing an Incrementally Constructible Code (ICC), an instance of LU-ECC.

[SSP13] achieves low asymptotic complexity for all data operations

- Polylog update cost.
- Constant read cost.
- Polylog audit cost.

All properties are achieved due to ICC.

Table of Contents

Proof of Retrievability

Incrementally Constructible Code

DPoR Scheme [SSP13]

Our DPoR Scheme: Porla

Incrementally Constructible Code[2]

Incrementally Constructible Code (ICC) is an $(2n, n, n)$-**linear** ECC, where n is number of data blocks.

- Encode n data blocks into $2n$ codeword blocks
- Knowledge of any n codeword blocks can recover n original data blocks

The **high level idea** is to build a hierarchical structure **H** over time time based on FFT when updating data blocks.

H has $L = \lceil \log_2 n \rceil$ levels as $\mathbf{H} = (\mathbf{H}_1, \mathbf{H}_2, \ldots, \mathbf{H}_L)$ where $\mathbf{H}_\ell$ contains the erasure codeword blocks of 2^ℓ most recently updated data blocks.

[2] http://www.cs.umd.edu/~elaine/docs/por.pdf

Building $\mathbf{H}_\ell$

Let $w = g^\alpha \mod p$ be a $2n$-th primitive root of unity mod p, where $p = \alpha 2n + 1$ is a prime for some $\alpha \in \mathbb{N}$.

When a block B is written at time t, compute its FFT code as

$$\mathbf{H}'_0.X \leftarrow B \qquad \mathbf{H}'_0.Y \leftarrow w^t \cdot B$$

- If $\mathbf{H}_0$ is empty, set $\mathbf{H}_0 \leftarrow \mathbf{H}'_0$
- Else, build $\mathbf{H}'_1$ as the FFT code for $(\mathbf{H}_0, \mathbf{H}'_0)$ as

$$\mathbf{H}'_1[0].X \leftarrow \mathbf{H}_0.X + w^{n/2}\mathbf{H}'_0.X \qquad \mathbf{H}'_1[0].Y \leftarrow \mathbf{H}_0.Y + w^{n/2}\mathbf{H}'_0.Y$$
$$\mathbf{H}'_1[1].X \leftarrow \mathbf{H}_0.X - w^{n/2}\mathbf{H}'_0.X \qquad \mathbf{H}'_1[1].Y \leftarrow \mathbf{H}_0.Y - w^{n/2}\mathbf{H}'_0.Y$$

  - If $\mathbf{H}_1$ is empty, set $\mathbf{H}_1 \leftarrow \mathbf{H}'_1$. Otherwise, continue to build FFT code for $(\mathbf{H}_1, \mathbf{H}'_1)$ and so on
  - Empty all levels being rebuilt.

Building $\mathbf{H}_\ell$

In general, when a data block B is written at time t, build FFT code $\mathbf{H}'_0$ for B as $\mathbf{H}'_0.X \leftarrow B \qquad \mathbf{H}'_0.Y \leftarrow w^t \cdot B$

- If $\mathbf{H}_0$ is empty, set $\mathbf{H}_0 \leftarrow \mathbf{H}'_0$.
- Else
  - $\mathbf{H}'_1 = \text{GenCode}(\mathbf{H}_0, \mathbf{H}'_0, i)$
  - For each successively non-empty levels i from 0 to $\ell - 1$:
    - $\mathbf{H}'_{i+1} = \text{GenCode}(\mathbf{H}_i, \mathbf{H}'_i, i)$
  - Set $\mathbf{H}_\ell \leftarrow \mathbf{H}'_\ell$ and empty $\mathbf{H}_0, \ldots, \mathbf{H}_{\ell-1}$

$\mathbf{H} \leftarrow \text{GenCode}(\mathbf{H}^0, \mathbf{H}^1, \ell)$:

1. Let $\nu = w^{n/2^\ell}$ be $2^{\ell+1}$-th primitive root of unity
2. For $i = 0$ to $2^\ell - 1$:
3. $\quad \mathbf{H}[i] \leftarrow \mathbf{H}^0[i] + \nu^i \cdot \mathbf{H}^1[i] \pmod p$
4. $\quad \mathbf{H}[i + 2^\ell] \leftarrow \mathbf{H}^0[i] - \nu^i \cdot \mathbf{H}^1[i] \pmod p$
5. Output $\mathbf{H}$

Building $\mathbf{H}_\ell$

The way $\mathbf{H}_\ell$ is built is like a butterfly network in FFT:

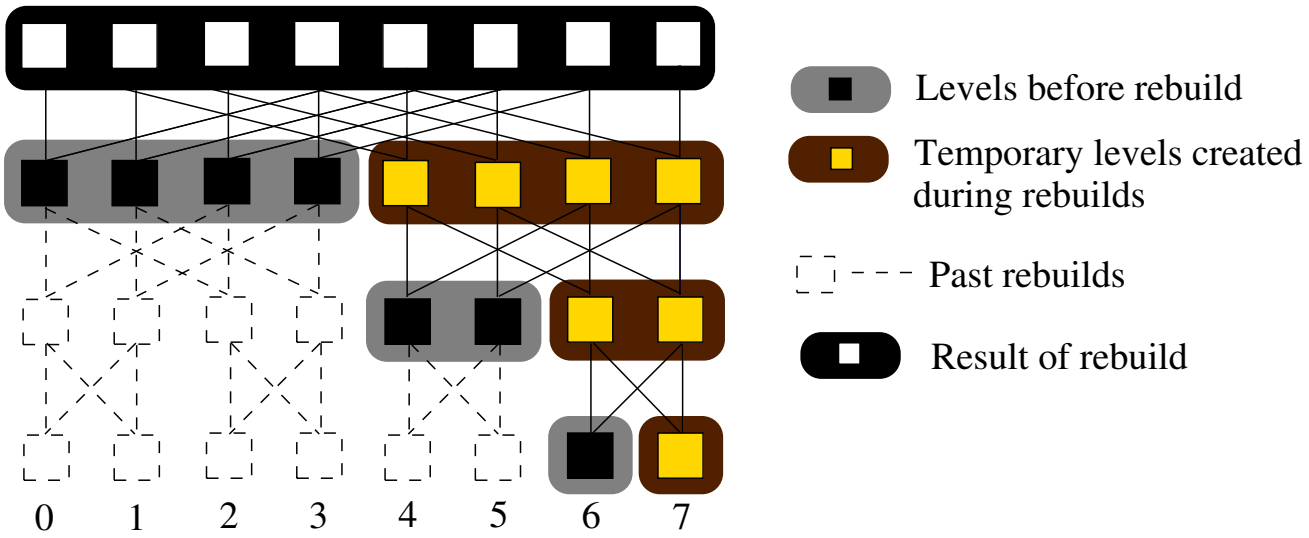

Figure 3: Butterfly network: the 8-th write operation encounters a full $\mathbf{H}_0$, $\mathbf{H}_1$, and $\mathbf{H}_2$. Blocks in $\mathbf{H}_0, \mathbf{H}_1, \mathbf{H}_2$ as well as the newly written block will be encoded and written to $\mathbf{H}_3$.

Building $\mathbf{H}_\ell$

Let t be the time when updating a block. Let $\mathbf{v}_\ell$ contains 2^ℓ data blocks being updated recently at time $t, t+1, \ldots, t+2^\ell-1$.

Using the strategy above, each $\mathbf{H}_\ell$ will have the following form:

$$\mathbf{H}_\ell = \pi_\ell(\mathbf{v}_\ell) \times [\mathbf{F}_\ell | \mathbf{D}_{\ell,t}\mathbf{F}_\ell] \tag{1}$$

where

- $\mathbf{F}_\ell := \mathrm{vand}(w^{\frac{2n}{2^\ell}\cdot 2^0}, w^{\frac{2n}{2^\ell}\cdot 2^1}, \ldots, w^{\frac{2n}{2^\ell}\cdot 2^{\ell-1}})$ is a $2^\ell \times 2^\ell$ FFT Vandermonde matrix
- $\mathbf{D}_{\ell,t} := \mathrm{diag}(w^{\psi_{\ell+1}(t)}, w^{\psi_{\ell+1}(t+1)}, \ldots, w^{\psi_{\ell+1}(t+n-1)})$ is a $2^\ell \times 2^\ell$ diagonal matrix
- w is the $2n$-th primitive root of unity mod p and $\psi(\cdot)$ denotes the bit reversal function.

Building $\mathbf{H}_\ell$

Lemma

Any $2^\ell \times 2^\ell$ submatrix of the generator matrix $\mathbf{G} := [\mathbf{F}_\ell \mid \mathbf{D}_{\ell,t}\mathbf{F}_\ell]$ is non-singular.

Proof.

I am unable to prove this! :(. That also means I am unable to determine the decoding algorithm for this ICC code. □

Table of Contents

Proof of Retrievability

Incrementally Constructible Code

DPoR Scheme [SSP13]

Our DPoR Scheme: Porla

DPoR Scheme [SSP13][3] — Audit Protocol

Basic protocol: for each level of $\ell \in [L]$, download λ random codeword blocks and check their integrity

- Communication cost: $O(\lambda B \log n)$

Improvement: Homomorphic MAC and random linear combination

$$\sigma \leftarrow \mathsf{MAC}_\kappa(B)$$

such that

$$\mathsf{MAC}_\kappa(B_i) + \mathsf{MAC}_\kappa(B_j) = \mathsf{MAC}_\kappa(B_i + B_j)$$

Random linear combination (with β as a random scalar):

$$B' \leftarrow \sum_{i=1}^{T} \beta^i B_i$$

[3] http://www.cs.umd.edu/~elaine/docs/por.pdf

DPoR Scheme [SSP13] — Audit Protocol

Improved protocol:

- Aggregate $\lambda \log n$ codeword blocks via rand. lin. comb.

$$B \leftarrow \sum_{i=1}^{L} \sum_{j=1}^{\lambda} \beta_{i,j} \mathbf{H}_i[x_j]$$

- Download the (encrypted) MAC of individual codeword blocks $\sigma_{i,j} = \mathrm{MAC}_\kappa(\mathbf{H}_i[x_j])$ and the aggregated block B
- Recompute the MAC of aggregated block

$$\sigma' \leftarrow \mathrm{MAC}_\kappa(B)$$

- Check if $\sigma' \stackrel{?}{=} \sum_{i=1}^{L} \sum_{j=1}^{\lambda} \sigma_{i,j}$

Communication cost: $O(\lambda^2 \log n + |B|)$, given $|\sigma| = O(\lambda)$

DPoR Scheme [SSP13] — Update Protocol

Update a block B follows the way how ICC code works.

- Build $\mathbf{H}_0'$ as the FFT code for B. Set $\mathbf{H}_0 \leftarrow \mathbf{H}_0'$ if $\mathbf{H}_0$ is empty. Else, compute FFT code for $\mathbf{H}_0, \mathbf{H}_0'$ as $\mathbf{H}_1'$ and update $\mathbf{H}_1 \leftarrow \mathbf{H}_1'$ if $\mathbf{H}_1$ is empty. Otherwise, continue the process to build a (larger) FFT code for level 1 and so on until reaching an empty level.
- It takes $O(B \cdot 2^\ell)$ time to compute FFT code for level ℓ.
- Each level ℓ is computed every 2^ℓ updates. Amortized server cost per update is $O(B \log n)$

DPoR Scheme [SSP13] — Update Protocol

Since each block has a MAC σ, when building FFT code, we need to build FFT code for its MAC σ as well.

Because σ is IND-CPA encrypted, server cannot perform the FFT computation directly on σ (MAC is homomorphic, but encrypted MAC is not homomorphic)

Instead, client will have to download all the tags in the levels to be built, decrypt them, and compute FFT on the tags, re-encrypt and finally upload the tags back to the server.

Armotized communication and client computation cost: $O(\lambda \log N)$

Table of Contents

Proof of Retrievability

Incrementally Constructible Code

DPoR Scheme [SSP13]

Our DPoR Scheme: Porla

Our Porla Scheme

Further improve **audit asymptotic cost** from SSP13 scheme

$$O(\lambda^2 \log n + |B|) \Rightarrow O(\lambda) \text{ or } O(\log B + \lambda)$$

Improve update cost as well (but requires some assumptions)

- Suitable to Cold Storage applications, where update exists but rarely happens.

High-level idea: Authenticated homomorphic polynomial commitment

Our improvement against SSP13 scheme:

- Transmit MAC of individual blocks $\Rightarrow$ Aggregate the authenticated commitments directly and only transmit the aggregated commitment.
- Transmit the aggreagated block $\Rightarrow$ prove the the knowledge of the aggregated block via **verifiable polynomial evaluation**

Authenticated Homomorphic Commitment

Let $b = (b_1, \ldots, b_k)$ be a codeword block with k chunks.
The (Pedersen) commitment of b is

$$\mathrm{cm} = \mathrm{Comm}(b) = \prod g_i^{b_i}$$

The <u>authenticated</u> commitment of b is

$$\mathrm{cm} = \mathrm{AComm}_\alpha(b) = h^r \cdot \mathrm{Comm}(b)^\alpha = h^r \prod g_i^{\alpha b_i}$$

where h, g_i are the generators, α is a secret and r is a random.

Homomorphic properties:

$$\mathrm{Comm}(b_i) \cdot \mathrm{Comm}(b_j) = \mathrm{Comm}(b_i + b_j)$$
$$\mathrm{AComm}_\alpha(b_i) \cdot \mathrm{AComm}_\alpha(b_j) = \mathrm{AComm}_\alpha(b_i + b_j)$$

Our Porla Scheme – Audit Protocol

- Aggregate $\lambda \log n$ codeword blocks and their authenticated commitments via rand. lin. comb.

$$B \leftarrow \sum_{i=1}^{L} \sum_{j=1}^{\lambda} \beta_{i,j} \cdot \mathbf{H}_i[x_j]$$

$$\mathrm{acm}_B \leftarrow \prod_{i=1}^{L} \prod_{j=1}^{\lambda} \beta_{i,j} \cdot \mathbf{H}_i[x_j].\mathrm{acm}$$

- Compute the commitment of B as $\mathrm{cm}_B \leftarrow \mathrm{Comm}(B)$
- Download acm_B and cm_B and check if $\mathrm{acm}_B \stackrel{?}{=} h^r \mathrm{cm}_B^{\alpha}$
- Given correct cm_B, execute verifiable polynomial evaluation protocol to attest B.

Our Porla Scheme – Update Protocol

Exploit the linearity of ICC code to update the authenticated commitment of the updated blocks directly on the server.

Recall how the codeword block is computed in ICC code:

$$B_i = B_i^0 + \nu^i B_i^1 \mod p$$
$$B_{i+2^\ell} = B_i^0 - \nu^i B_i^1 \mod p$$

The authenticated commitment can be done similarly due to homomorphism

$$\mathrm{acm}_i = \mathrm{acm}_i^0 \cdot \mathrm{acm}_i^{1^{\nu^i}}$$
$$\mathrm{acm}_{i+2^\ell} = \mathrm{acm}_i^0 \cdot \mathrm{acm}_i^{1^{-\nu^i}}$$

Need to handle some stuff to make it work. Details in paper.

Armotized client communication cost: $O(1)$ (with precomputation trick)

Open Research Questions

- Proof of Lemma in SSP13 Scheme
  - Ensure ICC code indeed works!
  - Critical to design a decoding algorithm for ICC code in case data is corrupted.
- Better Locally Updatable Code?
  - ICC expansion factor is $2\times \Rightarrow$ increase storage cost.
  - Need linearity property like ICC to keep the authenticated commitment work.